

GOOGLE ADK • A2A • PYTHON

Từ một AI Agent đến một hệ sinh thái Agent

Bản nguồn dành cho NotebookLM, Audio Overview và video ngắn

Một câu để nhớ: chatbot chủ yếu trả lời. Agent còn biết quyết định bước tiếp theo, gọi công cụ, quan sát kết quả và tiếp tục cho đến khi hoàn thành việc.

0. Cách dùng tài liệu này trong NotebookLM

Tài liệu này được viết thành các chương nhỏ, mỗi chương chỉ giữ một ý chính. Cách chia này giúp NotebookLM để tạo Audio Overview, Mind Map, câu hỏi ôn tập và kịch bản video ngắn.

Sau khi import, có thể hỏi:

- “Giải thích toàn bộ hệ thống cho người chưa biết lập trình trong 5 phút.”
- “Tạo podcast hai người nói về quá trình nâng cấp từ Trip Planner đến A2A Orchestrator.”
- “Biến chương Dashboard Insights thành 6 video ngắn, mỗi video 45 giây.”
- “So sánh SequentialAgent, ParallelAgent và LoopAgent bằng ví dụ đời thường.”
- “Chỉ dùng thông tin trong nguồn, viết phần Project Experience cho CV intern.”

NotebookLM dùng nội dung trong nguồn để trả lời. “Nguồn” ở đây nghĩa là file bạn upload. Vì vậy tài liệu luôn phân biệt ba mức:

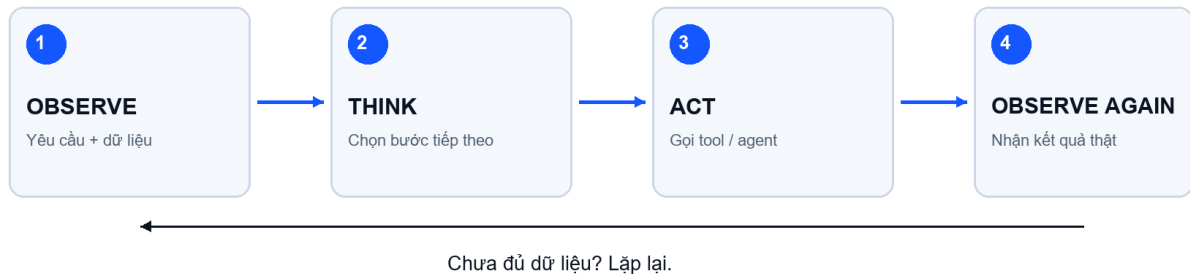
- Đã có trong code: có thể chỉ đến file hoặc cấu trúc thật.
- Ý đồ thiết kế: điều project muốn chứng minh.
- Đề xuất nâng cấp: chưa phải tính năng đã hoàn thành.

Phân biệt ba mức này rất quan trọng. Portfolio tốt không chỉ kể mình đã làm gì. Portfolio tốt còn cho thấy mình biết phần nào đang thiếu.

1. AI Agent thật ra là gì?

Vòng lặp của một AI Agent

Model quyết định. Tool thực thi. Kết quả quay lại làm dữ liệu mới.



Không có hành động và quan sát lại, đó mới chỉ là một lượt hỏi–đáp.

Hình — 1. AI Agent thật ra là gì?

Một mô hình ngôn ngữ, hay LLM, là chương trình có khả năng đọc và tạo ngôn ngữ. Nếu chỉ đưa câu hỏi vào LLM rồi lấy câu trả lời ra, ta có một luồng hỏi–đáp.

Một AI Agent thêm vào luồng đó khả năng tự chọn hành động. Hành động có thể là gọi hàm Python, đọc file, tìm kiếm, tính toán, lưu dữ liệu hoặc chuyển việc cho một agent khác.

Agent chạy theo một vòng lặp:

1. Observe — Quan sát: nhận yêu cầu của người dùng và dữ liệu hiện có.
2. Think — Suy nghĩ: mô hình xác định đang thiếu gì và nên làm bước nào.
3. Act — Hành động: agent gọi một tool hoặc chuyển việc.
4. Observe again — Quan sát lại: kết quả hành động quay về làm dữ liệu mới.
5. Lặp lại cho đến khi đủ thông tin để trả lời.

Ví dụ:

Người dùng hỏi: “Lên kế hoạch đi Đà Nẵng ba ngày và nhớ lịch trình này.”

Chatbot có thể viết một lịch trình rồi dừng. Trip Planner Agent vừa viết lịch trình, vừa gọi tool lưu dữ liệu vào trạng thái phiên làm việc. Lần sau người dùng hỏi “Lịch trình cũ của tôi đâu?”, agent gọi tool đọc lại.

Điểm khác nằm ở động từ làm.

2. Bốn lớp của một project AI Agent

2.1. Lớp não và vòng lặp

Đây là nơi agent quyết định bước tiếp theo. Trong một project thuần Python, phần này có thể nằm trong thư mục `agent_core/`. Trong Google ADK, phần lớn cấu hình nằm trong `agent.py`.

Google ADK, viết đầy đủ là Agent Development Kit, là bộ khung giúp khai báo model, instruction, tools, workflow, session và cách chạy agent. “Bộ khung” nghĩa là nó cung cấp sẵn cấu trúc để ta không phải tự viết toàn bộ vòng lặp từ đầu.

2.2. Lớp công cụ

Tool là một chức năng mà agent có thể gọi. Ví dụ:

- `save_trip_detail()` lưu lịch trình.
- `read_local_file()` đọc nội dung file.
- `search_wikipedia()` lấy dữ kiện World Cup.
- `search_knowledge_base()` tìm đoạn dữ liệu tương tự trong kho kiến thức.

LLM không trực tiếp chạy Python. Nó chỉ chọn tên tool và tạo bộ tham số. Runtime, tức phần chương trình đang vận hành agent, mới gọi hàm thật.

2.3. Lớp cấu hình

Đây là nơi giữ API key, tên model và biến môi trường.

`.env` là file chứa các giá trị cấu hình riêng của máy, thường có API key. API key là chuỗi bí mật dùng để xác thực khi gọi dịch vụ model. Không đưa `.env` lên GitHub.

`.env.example` là bản mẫu chỉ có tên biến, không có bí mật. Người khác nhìn file này sẽ biết cần tự tạo những giá trị gì.

`.gitignore` nói với Git những file nào không được theo dõi, ví dụ `.env`, thư mục cache và database local.

2.4. Lớp giao diện

`app.py` thường là điểm vào cho giao diện, ví dụ Streamlit. Streamlit là thư viện biến code Python thành web app đơn giản.

Với Google ADK, ta có thể dùng `adk web` để mở giao diện thử nghiệm hoặc `adk api_server` để đưa agent ra dạng API. API là cổng để một chương trình khác gửi yêu cầu và nhận phản hồi.

Chia đúng lớp. Sau này đỡ rối.

3. Bộ file tối thiểu và vai trò của từng file

```
my_agent/
├── agent_core/
│   ├── brain.py
│   ├── registry.py
│   └── tools.py
├── scripts/
│   └── smoke_test.py
├── .env
├── .env.example
├── .gitignore
├── app.py
├── README.md
├── requirements.txt
└── run.ps1
```

brain.py

Chứa vòng lặp điều khiển: gửi prompt cho model, đọc tool call, gọi tool, trả kết quả về model.

“Prompt” là nội dung gửi cho model. “Tool call” là yêu cầu có cấu trúc do model tạo ra, ví dụ tên tool là calculator và tham số là {"expression": "2+2"}.

registry.py

Registry là sổ đăng ký tool. Nó biết tool nào tồn tại, mô tả của tool, tham số cần có và hàm Python nào thực thi.

Một ToolSpec cơ bản thường có:

- name: tên tool để model gọi.
- description: tool làm gì và khi nào nên dùng.
- parameters: schema mô tả dữ liệu đầu vào.
- func: hàm Python thật.

Schema là bản mô tả cấu trúc dữ liệu. JSON Schema có thể nói “tham số amount phải là số” hoặc “currency phải là chuỗi”. Nhờ đó runtime có thể chặn dữ liệu sai trước khi gọi hàm.

tools.py

Chứa phần tay chân. Ví dụ:

- Calculator dùng cây cú pháp an toàn, chỉ cho phép toán tử hợp lệ.
- Currency converter dùng bảng tỉ giá offline.
- PDF reader trích xuất chữ và giới hạn độ dài phản hồi.

Không dùng eval() trực tiếp cho máy tính. eval() có thể chạy chuỗi như code Python, nên một biểu thức tưởng là phép tính có thể biến thành lệnh nguy hiểm. Cách an toàn hơn là parse biểu thức thành AST, tức cây biểu diễn cú pháp, rồi chỉ cho phép các node số học.

smoke_test.py

Smoke test là bài kiểm tra nhanh xem phần quan trọng có chạy hay không. Hãy test tool trước khi nối model.

Nếu calculator sai, lỗi nằm ở tool. Nếu calculator đúng nhưng agent gọi sai, lỗi nằm ở instruction hoặc schema. Tách lớp giúp tìm lỗi nhanh.

requirements.txt

Liệt kê thư viện cần cài. Ví dụ:

```
streamlit  
python-dotenv  
google-gemini  
pypdf
```

python-dotenv đọc biến từ .env. pypdf đọc nội dung PDF. google-gemini là SDK, tức bộ thư viện giúp Python gọi dịch vụ Gemini.

README.md

Giải thích project làm gì, cài ra sao, chạy lệnh nào, có giới hạn gì. README là cửa vào cho người đọc GitHub.

run.ps1

PowerShell script gom các lệnh chạy trên Windows. Thay vì nhớ nhiều lệnh, người dùng chạy một file.

Một câu để nhớ: file nào cũng phải trả lời được “tại sao nó tồn tại?”.

4. Giải phẫu một agent.py trong Google ADK

Một agent cơ bản có thể được khai báo như sau:

```
root_agent = Agent(  
    name="trip_planner_agent",  
    model="gemini-3.1-flash-lite-preview",  
    description="Lập kế hoạch chuyến đi",  
    instruction="Hỏi đủ thông tin, tạo lịch trình và dùng tool để lưu.",  
    tools=[save_trip_detail, get_trip_details],  
)
```

root_agent

Đây là biến mà ADK tìm để biết agent chính của project là ai. “Root” nghĩa là điểm bắt đầu của cây xử lý.

name

Tên định danh trong hệ thống. Tên nên ngắn, không dấu cách và thể hiện vai trò.

model

Model là bộ máy ngôn ngữ agent dùng để suy luận và tạo câu trả lời. Model khác tool: model quyết định, tool thực thi.

description

Mô tả ngắn về khả năng của agent. Description có thể được agent điều phối dùng để chọn đúng chuyên gia. Nếu description mơ hồ, routing cũng dễ sai.

instruction

Instruction là quy tắc làm việc lâu dài: agent phải hỏi gì, dùng tool khi nào, định dạng đầu ra ra sao, điều gì không được làm.

Instruction khác câu hỏi của người dùng. Người dùng nói “hãy làm X”; instruction nói “khi làm bất cứ việc gì, hãy tuân theo Y”.

tools

Danh sách hành động agent được phép gọi. Không đưa tool thừa. Tool càng nhiều và mô tả càng giống nhau, model càng khó chọn.

output_key

Một số agent lưu kết quả cuối vào một khóa trong session state. Khóa giống nhãn trên một ngăn kéo. Agent sau đọc đúng nhãn để lấy đúng dữ liệu.

sub_agents

Danh sách agent con có chuyên môn riêng. Root agent có thể chuyển quyền xử lý sang một agent con.

5. Một tool chạy như thế nào?

Giả sử có hàm:

```
def save_trip_detail(key: str, value: str, tool_context: ToolContext) -> dict:
    tool_context.state[key] = value
    return {
        "status": "success",
        "saved_key": key,
        "saved_value": value,
    }
```

def là từ khóa tạo hàm Python. Hàm là một khối thao tác có tên, nhận dữ liệu vào và trả dữ liệu ra.

key: str nghĩa là tên trường dự kiến là chuỗi, ví dụ destination hoặc budget. value: str là giá trị cần lưu. Dấu -> dict nói kết quả dự kiến là dictionary, tức tập các cặp khóa–giá trị.

ToolContext là ngữ cảnh ADK đưa vào khi tool chạy. Nó cho tool truy cập state và thông tin của lượt làm việc.

Luồng thật:

1. ADK đọc tên hàm, mô tả và type hint.
2. Model thấy tool có thể lưu lịch trình.
3. Model tạo tool call với key và value.
4. Runtime kiểm tra tham số rồi gọi hàm Python.
5. Hàm ghi vào tool_context.state.
6. Hàm trả trạng thái thành công cùng khóa và giá trị vừa lưu.
7. Kết quả quay lại model.
8. Model nói với người dùng rằng lịch trình đã được lưu.

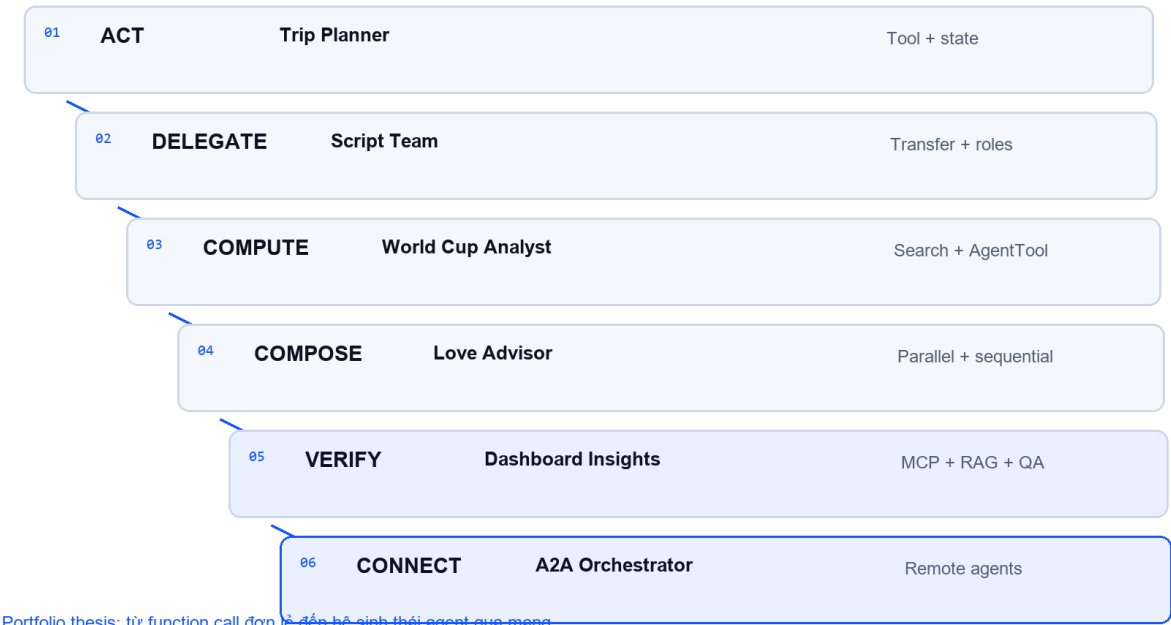
Model không “tưởng tượng” rằng tool đã chạy. Runtime phải thực sự chạy nó.

Sai ở đâu, trả lời ngay ở đó.

6. Ảnh xạ khung cơ bản sang sáu project thật

Sáu nấc năng lực kiến trúc

Mỗi project thêm một lớp còn thiếu — không phải sáu chatbot giống nhau.



Hình — 6. Ảnh xạ khung cơ bản sang sáu project thật

Khái niệm cơ bản	Cách xuất hiện trong hệ project
Brain/loop	Agent, LlmAgent, SequentialAgent, ParallelAgent, LoopAgent
Tool registry	Danh sách tools=[...], AgentTool, MCP Toolset
State	ToolContext.state, output_key, session service
Memory	In-memory memory service hoặc kho dữ liệu/RAG
Interface	adk web, adk api_server --a2a, A2A endpoint
Config	.env, model name, requirements
Test	test script, testcase file, local runner
Distributed agent	RemoteA2aAgent đọc Agent Card và gọi agent ở port khác

Sáu project không lặp lại cùng một bài. Chúng tăng dần độ khó kiến trúc.

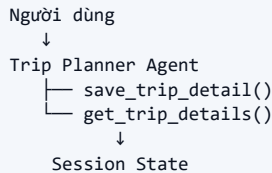
7. Project 1 — Trip Planner

Ý tưởng

Trip Planner là ví dụ nền tảng: một agent có hai tool và biết nhớ dữ liệu trong phiên.

Người dùng đưa điểm đến, thời gian, sở thích và ngân sách. Agent tạo lịch trình. Sau đó agent lưu lịch trình để truy xuất lại trong cùng phiên làm việc.

Kiến trúc



Đây là single-agent architecture, nghĩa là chỉ một agent chịu trách nhiệm suy luận và giao tiếp.

Hai tool

save_trip_detail

Nhận dữ liệu chuyến đi và ghi vào state. State là vùng nhớ tạm gắn với phiên hội thoại.

get_trip_details

Đọc dữ liệu đã lưu từ state. Nếu không có dữ liệu, tool trả thông báo rõ thay vì tạo ra lịch trình giả.

Session và state khác nhau thế nào?

Session là cả cuộc hội thoại: ai nói gì, tool nào đã gọi, state hiện tại là gì. State chỉ là vùng dữ liệu có cấu trúc bên trong session.

Trong test local, project dùng InMemorySessionService. “In-memory” nghĩa là dữ liệu nằm trong RAM. Tắt chương trình thì dữ liệu mất. Cách này phù hợp để học và test, chưa phù hợp lưu lâu dài.

Luồng chạy

1. Khởi động agent.
2. Tạo session mới.
3. Người dùng yêu cầu lịch trình.
4. Agent hỏi thêm nếu thiếu dữ liệu quan trọng.
5. Agent tạo kế hoạch.
6. Agent gọi save_trip_detail.
7. Người dùng hỏi lại.
8. Agent gọi get_trip_details.

Điểm khác biệt

Project chứng minh ranh giới giữa “trả lời” và “tạo thay đổi có thể truy xuất”. Tool không chỉ lấy dữ liệu. Tool còn cập nhật trạng thái.

Hạn chế trung thực

- Dữ liệu trong RAM mất khi restart.
- Chưa có API thời tiết, bản đồ, giá vé hoặc khách sạn.
- Chưa có schema mạnh cho toàn bộ trip object.
- Chưa có xác nhận trước các hành động có chi phí thật.

Nâng cấp hợp lý

- Dùng DatabaseSessionService để dữ liệu tồn tại sau restart.
- Tách tool tìm chuyến bay, thời tiết và bản đồ.
- Thêm validation ngân sách và múi giờ.
- Tạo artifact xuất lịch trình thành PDF hoặc calendar.

Câu chốt

Trip Planner dạy bài đầu tiên: agent muốn hữu ích phải có tay để làm và chỗ để nhớ.

8. Project 2 — Script Team

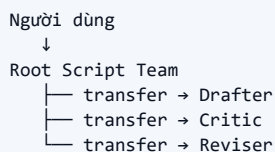
Ý tưởng

Script Team biến một nhiệm vụ viết kịch bản thành một đội có vai trò:

- Drafter viết bản đầu.
- Critic đánh giá.
- Reviser sửa theo feedback.
- Root agent điều phối.

Mỗi agent không cố làm tất cả. Mỗi agent làm một việc rõ.

Kiến trúc chuyển giao



Transfer nghĩa là agent hiện tại chuyển quyền xử lý lượt hội thoại cho agent khác. Đây khác với gọi tool: agent con trở thành người xử lý chính của bước đó.

PlanReActPlanner

Planner là thành phần giúp model lập kế hoạch rồi hành động theo từng bước. “ReAct” ghép từ Reason và Act: suy luận rồi hành động, quan sát kết quả và tiếp tục.

Planner hữu ích khi nhiệm vụ có chuỗi bước. Tuy nhiên planner không đảm bảo mọi bước đều đúng; vẫn cần tool rõ, output rõ và kiểm tra.

read_local_file

Tool đọc file local để agent dùng brief, notes hoặc tài liệu nguồn.

Project giới hạn nội dung khoảng 20.000 ký tự. Giới hạn này tránh đẩy file quá lớn vào context. Context là lượng thông tin model đang nhìn thấy trong một lượt. Nếu context quá dài, chi phí tăng và phần quan trọng dễ bị loãng.

Output key

Drafter, Critic và Reviser có thể ghi kết quả vào các khóa khác nhau. Agent sau lấy đúng sản phẩm của agent trước.

Đây là cách tạo một blackboard workflow đơn giản: các vai trò viết kết quả lên một bảng chung có nhãn.

Structured feedback với Pydantic

ScriptFeedback là model dữ liệu Pydantic. Pydantic là thư viện kiểm tra kiểu và cấu trúc dữ liệu Python.

Thay vì critic trả một đoạn văn tùy ý, feedback có thể gồm trường cụ thể như:

- điểm mạnh,
- vấn đề,
- mức độ,
- đề xuất sửa.

Output có cấu trúc dễ truyền cho Reviser và dễ hiển thị trên giao diện.

Safety callback

Callback là hàm được runtime gọi ở một thời điểm xác định. Project có callback kiểm tra từ cấm trước hoặc quanh quá trình model xử lý.

Callback khác instruction. Instruction là lời dẫn cho model; callback là code bắt buộc chạy. Với quy tắc an toàn quan trọng, code thường đáng tin hơn lời nhắc đơn thuần.

Memory và session

Project thử cả InMemoryMemoryService và DatabaseSessionService.

Memory là kho kiến thức có thể được gọi lại giữa các cuộc hội thoại. Session là một cuộc hội thoại cụ thể. Nói ngắn:

- Session nhớ “đang nói chuyện gì”.
- Memory nhớ “trước đây đã biết gì”.

Luồng chạy

1. Người dùng đưa brief hoặc đường dẫn file.
2. Root agent đọc nguồn khi cần.
3. Drafter tạo bản nháp.
4. Critic tạo feedback có cấu trúc.
5. Reviser đọc draft và feedback rồi sửa.
6. Root agent trả bản cuối.

Điểm khác biệt

Script Team dạy cách chuyên môn hóa bằng chuyển giao. Chất lượng đến từ việc phân vai, không chỉ đổi prompt.

Hạn chế trung thực

- Transfer dựa nhiều vào mô tả vai trò và quyết định của model.
- Nếu output key hoặc instruction không khớp, agent sau có thể không lấy đúng dữ liệu.
- Chưa có thước đo tự động về độ dài, cấu trúc cảnh hoặc tính nhất quán nhân vật.
- File dài bị cắt, nên phần sau giới hạn có thể không được đọc.

Nâng cấp hợp lý

- Thêm evaluator chấm rubric trước khi kết thúc.
- Tách source ingestion khỏi writing workflow.
- Dùng artifact cho từng phiên bản draft.
- Thêm version history và diff giữa bản nháp với bản sửa.

Câu chốt

Đừng làm một agent thật dài. Hãy chia vai để mỗi instruction thật rõ.

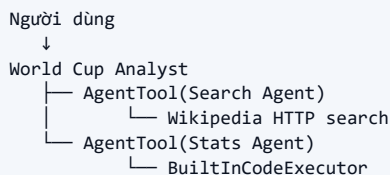
9. Project 3 — World Cup Analyst

Ý tưởng

World Cup Analyst trả lời câu hỏi bóng đá bằng cách kết hợp:

- tìm kiếm dữ kiện từ Wikipedia,
- tính thống kê bằng code,
- viết báo cáo phân tích.

Kiến trúc agent dùng agent như tool



AgentTool bọc một agent con thành tool. Root agent vẫn giữ quyền điều khiển; agent con được gọi như một năng lực chuyên biệt rồi trả kết quả về.

Đây khác Script Team:

- Script Team chuyển quyền cho agent con.
- World Cup Analyst gọi agent con như tool rồi nhận kết quả trở lại.

Wikipedia search qua HTTP

HTTP là giao thức phổ biến để chương trình giao tiếp qua web. Tool gửi request tới API hoặc trang dữ liệu và nhận response.

Tool cần xử lý:

- query hợp lệ,
- timeout,
- lỗi mạng,
- kết quả rỗng,
- nội dung không đáng tin.

Wikipedia phù hợp demo tra cứu kiến thức tổng quan, nhưng không phải nguồn chính thức cho mọi thống kê. Với portfolio, cần nói rõ phạm vi nguồn.

BuiltInCodeExecutor

Code executor cho phép agent tạo và chạy code để tính toán. Nó hữu ích cho tổng, trung bình, tỉ lệ hoặc so sánh bảng số liệu.

Điểm mạnh: model không cần tự nhẩm số.

Điểm rủi ro: môi trường chạy code cần được giới hạn và giám sát. Không nên để code tùy ý truy cập toàn máy.

last_wc_report

Kết quả báo cáo cuối được lưu vào output key. Agent khác hoặc lượt sau có thể lấy lại báo cáo theo khóa này.

Luồng chạy

1. Người dùng hỏi một vấn đề World Cup.
2. Root agent tách nhu cầu thành dữ kiện và phép tính.

3. Search Agent lấy nguồn liên quan.
4. Stats Agent tính toán khi cần.
5. Root agent tổng hợp, nêu nguồn và kết luận.

Điểm khác biệt

Project thể hiện mô hình delegation as a tool: agent con là một khả năng được gọi có kiểm soát.

Hạn chế trung thực

- Nguồn Wikipedia không đủ cho báo cáo nghiên cứu chuyên sâu.
- Chưa có pipeline kiểm tra chéo nhiều nguồn.
- Code executor cần sandbox rõ nếu đưa lên production.
- Tài liệu nhắc dependency nhưng thư mục project hiện không có `requirements.txt`; đây là độ lệch giữa docs và source cần sửa.

Nâng cấp hợp lý

- Thêm nguồn FIFA hoặc dataset thống kê chính thức.
- Gắn citation cho từng claim.
- Tạo schema báo cáo gồm question, evidence, calculation và confidence.
- Thêm unit test cho tool tìm kiếm và phép tính.

Câu chốt

Model giải ngôn ngữ. Dữ kiện để tool lấy. Số liệu để code tính.

10. Project 4 — Love Advisor

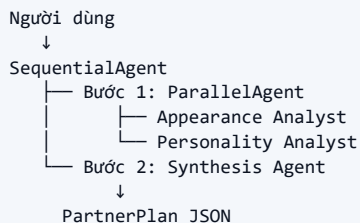
Ý tưởng

Love Advisor phân tích hai luồng độc lập:

- hình mẫu ngoại hình,
- hình mẫu tính cách.

Hai luồng chạy song song. Sau đó một agent tổng hợp tạo kế hoạch bạn đời có cấu trúc.

Kiến trúc song song rồi tuần tự



ParallelAgent chạy các nhánh độc lập cùng lúc. “Song song” không nhất thiết nghĩa là mọi dòng code luôn chạy đúng cùng một nanosecond; ý chính là các nhánh không phải chờ kết quả của nhau.

SequentialAgent chạy theo thứ tự. Bước sau chỉ bắt đầu khi bước trước hoàn thành.

Kết hợp hai cấu trúc giúp giảm thời gian ở phần độc lập và vẫn giữ thứ tự ở phần tổng hợp.

DuckDuckGo và academic frameworks

Agent có công cụ tìm kiếm web và một số khung kiến thức được nhúng trong instruction hoặc code.

Search cho dữ liệu bên ngoài. Khung kiến thức cho tiêu chí phân tích. Hai thứ không giống nhau:

- Search trả thông tin mới.
- Framework cho cách suy nghĩ về thông tin.

Structured output với PartnerPlan

PartnerPlan là Pydantic model lồng nhau. “Lồng nhau” nghĩa là một trường có thể chứa một object có cấu trúc khác.

Ví dụ:

```

{
  "appearance_profile": {},
  "personality_profile": {},
  "compatibility_notes": [],
  "action_plan": []
}
  
```

JSON là định dạng dữ liệu dạng khóa–giá trị, phù hợp để giao diện đọc và hiển thị thành từng phần.

Tone là một quyết định sản phẩm

Instruction hiện dùng giọng mạnh và có phần công kích. Đây là một thử nghiệm persona, không phải chuẩn mặc định nên sao chép.

Persona là tính cách giao tiếp được đặt cho agent. Persona có thể làm sản phẩm dễ nhớ, nhưng không được làm giảm an toàn, tôn trọng hoặc độ tin cậy.

Đưa lên portfolio nên trình bày trung thực:

- Giọng mạnh là thử nghiệm về engagement.
- Cần thêm tone mode: supportive, direct, neutral.

- Cần disclaimer vì đây không phải tư vấn y khoa hoặc tâm lý chuyên nghiệp.

Luồng chạy

1. Nhận mô tả hoặc yêu cầu của người dùng.
2. Hai analyst xử lý độc lập.
3. Mỗi analyst lưu output vào state.
4. Synthesis Agent đọc cả hai.
5. Kết quả được ép theo schema PartnerPlan.
6. Giao diện có thể render từng section thay vì một đoạn văn dài.

Điểm khác biệt

Project chứng minh workflow composition: ghép song song, tuần tự và structured output trong một pipeline.

Hạn chế trung thực

- Chủ đề nhạy cảm, dễ tạo kết luận quá chắc chắn.
- Search web có thể đưa nguồn kém chất lượng.
- Persona mạnh có thể làm tổn thương người dùng.
- Schema đẹp không đồng nghĩa nội dung luôn đúng.

Nâng cấp hợp lý

- Thêm nguồn, confidence và disclaimer.
- Cho người dùng chọn tone.
- Thêm guardrail phát hiện nội dung quấy rối, tự hại hoặc lạm dụng.
- Không suy luận đặc điểm nhạy cảm từ ảnh hoặc dữ liệu thiếu.

Câu chốt

Chạy song song để nhanh. Tổng hợp tuần tự để không mất mạch. Schema để kết quả dùng được.

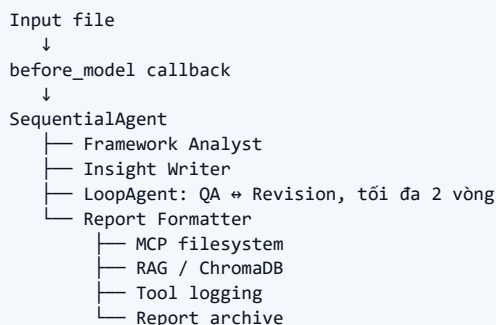
11. Project 5 — Dashboard Insights

Ý tưởng

Dashboard Insights nhận dữ liệu kinh doanh từ file, phân tích framework, viết insight, tự kiểm tra, sửa tối đa hai vòng và định dạng báo cáo cuối.

Đây là project có nhiều lớp kỹ thuật nhất trong nhóm chuyên gia.

Kiến trúc



Ingestion — đưa dữ liệu vào hệ thống

`extract_file_content` đọc nội dung từ CSV, XLSX hoặc DOCX.

- CSV là file bảng dạng văn bản, mỗi dòng là một bản ghi.
- XLSX là workbook Excel.
- DOCX là tài liệu Word.

`before_model_callback` chạy trước khi model nhận prompt. Callback lấy đường dẫn file hoặc upload, trích xuất nội dung rồi gắn vào context.

Nội dung được giới hạn khoảng 20.000 ký tự. Đây là một biện pháp kiểm soát context, nhưng có thể làm mất phần dữ liệu ở cuối file. Với dataset lớn, nên tóm tắt theo chunk thay vì cắt cứng.

MCP filesystem

MCP, viết đầy đủ là Model Context Protocol, là chuẩn để agent kết nối đến công cụ hoặc nguồn dữ liệu bên ngoài qua một giao diện thống nhất.

Project dùng filesystem MCP server với các thao tác như list và read file trong thư mục dữ liệu được cho phép.

MCP không phải A2A:

- MCP nối agent với tool hoặc data.
- A2A nối agent với agent.

Một câu để nhớ: MCP là ổ cắm cho công cụ. A2A là ngôn ngữ giữa đồng nghiệp.

RAG với ChromaDB

RAG, viết đầy đủ là Retrieval-Augmented Generation, là cách tìm các đoạn dữ liệu liên quan rồi đưa chúng vào context trước khi model trả lời.

Luồng RAG:

1. Chia tài liệu thành đoạn.
2. Biến mỗi đoạn thành embedding.
3. Lưu embedding vào vector database.
4. Khi có câu hỏi, biến câu hỏi thành embedding.

5. Tìm các vector gần nhất.
6. Đưa đoạn tương ứng cho model.

Embedding là dãy số biểu diễn ý nghĩa của văn bản. Hai đoạn gần nghĩa thường có vector gần nhau.

ChromaDB là vector database local. sentence-transformers tạo embedding.

Project có custom tool để lưu và tìm tri thức. Điều này giúp agent tham chiếu insight hoặc báo cáo trước đó.

Framework Analyst

Agent đầu xác định góc nhìn phân tích: funnel, retention, cohort, vận hành hoặc tài chính tùy dữ liệu.

Framework là khung câu hỏi giúp không phân tích lan man. Ví dụ funnel hỏi người dùng rơi ở bước nào; cohort hỏi nhóm người dùng theo thời điểm có hành vi khác nhau ra sao.

Insight Writer

Biến quan sát thành insight. Insight tốt thường có bốn phần:

1. Điều gì xảy ra.
2. Bằng chứng nào cho thấy điều đó.
3. Tại sao có thể xảy ra.
4. Nên làm gì tiếp theo.

Phần “tại sao” phải được gắn nhãn giả thuyết nếu dữ liệu chưa chứng minh nguyên nhân.

LoopAgent cho QA và revision

LoopAgent lập một nhóm bước đến khi đạt điều kiện hoặc chạm giới hạn.

Trong project, QA kiểm tra báo cáo, Revision sửa và vòng lặp tối đa hai lần. `max_iterations=2` là van an toàn tránh agent tự sửa mãi.

Vòng lặp chất lượng chỉ có giá trị khi rubric rõ. Nếu QA chỉ nói “hãy tốt hơn”, revision dễ thay chữ mà không tăng độ đúng.

Tool logging

`after_tool_callback` ghi log sau mỗi lần tool chạy. Log dạng JSONL: mỗi dòng là một object JSON.

Logging giúp trả lời:

- Tool nào được gọi?
- Tham số gì?
- Thành công hay lỗi?
- Mất bao lâu?

Đây là nền tảng của observability. Observability là khả năng nhìn vào hệ thống đang chạy và hiểu chuyện gì đã xảy ra.

Report archive

Kết quả được lưu để truy xuất lại. Đây là bước tiến từ câu trả lời tạm thời sang artifact có vòng đời.

Artifact là sản phẩm tạo ra bởi agent, ví dụ report, file hoặc dữ liệu có cấu trúc.

Luồng chạy

1. Người dùng chọn file.
2. Callback trích nội dung.
3. Framework Analyst chọn cách đọc dữ liệu.
4. Insight Writer tạo báo cáo.
5. QA Agent chấm theo tiêu chí.

6. Revision Agent sửa nếu cần.
7. Lập tối đa hai vòng.
8. Formatter chuẩn hóa đầu ra.
9. Tool callback ghi dấu vết.
10. Báo cáo được archive và có thể đưa vào RAG.

Điểm khác biệt

Dashboard Insights kết hợp workflow, ingestion, MCP, RAG, loop QA, logging và archive. Nó gần một pipeline ứng dụng hơn là một prompt demo.

Hạn chế trung thực

- Archive dùng cùng period có thể ghi đè bản cũ.
- Kho RAG chung có thể làm insight của domain này lẫn sang domain khác.
- Generated report đôi khi đưa giả thuyết như kết luận.
- Cắt 20.000 ký tự có thể bỏ dữ liệu.
- Tài liệu và đường dẫn Chroma có dấu hiệu chưa đồng bộ hoàn toàn.
- Chưa có semantic validation đối chiếu từng số trong báo cáo với file nguồn.

Nâng cấp hợp lý

- Dùng report_id duy nhất thay vì chỉ period.
- Tách collection RAG theo tenant, project và domain.
- Thêm citation đến sheet/cell hoặc row nguồn.
- Dùng code để tính metric trước, model chỉ diễn giải.
- Thêm evaluator kiểm tra claim–evidence.
- Thêm privacy policy và xóa dữ liệu theo yêu cầu.

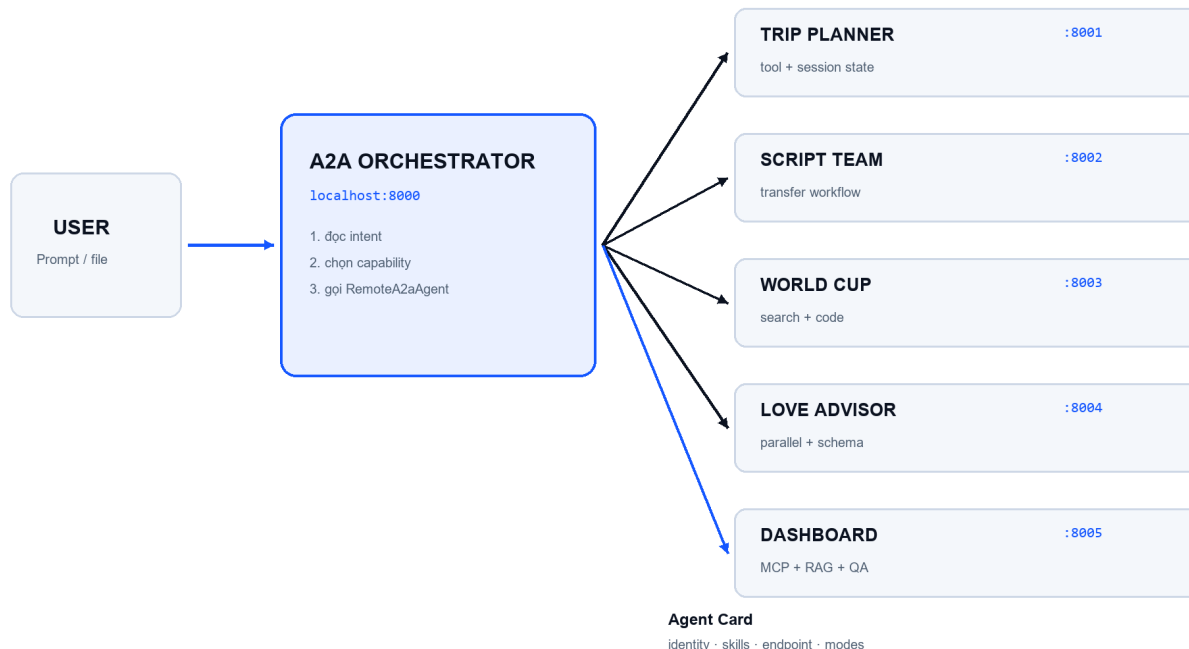
Câu chốt

Một agent phân tích đáng tin không chỉ viết hay. Nó phải chỉ ra số lấy từ đâu, đã kiểm tra thế nào và còn nghi ngờ điều gì.

12. Project 6 — A2A Orchestrator

A2A Orchestrator — luồng request xuyên sáu service

Agent Card cho biết gọi ai. A2A chuyển message. Specialist tự chạy workflow nội bộ.



MCP nối agent với tool/data. A2A nối agent với agent.

Hình — 12. Project 6 — A2A Orchestrator

Ý tưởng

Năm project chuyên gia chạy như năm dịch vụ độc lập:

- Trip Planner ở port 8001.
- Script Team ở port 8002.
- World Cup Analyst ở port 8003.
- Love Advisor ở port 8004.
- Dashboard Insights ở port 8005.

Orchestrator chạy tại port 8000 và định tuyến yêu cầu đến đúng chuyên gia.

Port là số cổng trên máy. Cùng một máy có thể chạy nhiều server nếu mỗi server dùng một port khác.

Kiến trúc

```

    ↗ Trip Planner :8001
Người dùng → Orchestrator:8000 → Script Team :8002
    ↘ World Cup Analyst :8003
    ↘ Love Advisor :8004
    ↘ Dashboard Insights :8005
  
```

RemoteA2aAgent

Đây là đại diện local cho một agent đang chạy ở server khác. Orchestrator nhìn nó như sub-agent, nhưng khi gọi, request đi qua mạng.

“Remote” không nhất thiết là máy ở quốc gia khác. Localhost khác port cũng đã là hai service giao tiếp qua mạng.

A2A là gì?

A2A, viết đầy đủ là Agent2Agent Protocol, là chuẩn để các agent độc lập tìm hiểu khả năng của nhau, gửi message, quản lý task và nhận artifact.

A2A cho phép agent hợp tác mà không cần biết code nội bộ, memory hoặc tool của nhau.

Agent Card

Agent Card là hồ sơ năng lực dạng JSON. Nó mô tả:

- tên agent,
- agent làm gì,
- skills,
- input/output hỗ trợ,
- endpoint,
- khả năng streaming hoặc cơ chế khác,
- yêu cầu xác thực.

Orchestrator đọc card để biết “đồng nghiệp này làm được gì và gọi ở đâu”.

Description và Agent Card là hợp đồng routing. Viết sai hợp đồng, gọi sai người.

Message, Task và Artifact

- Message: một lượt giao tiếp giữa client và agent.
- Task: đơn vị công việc có ID và trạng thái.
- Artifact: kết quả tạo ra, ví dụ báo cáo hoặc file.

Task phù hợp với việc dài hoặc bất đồng bộ. “Bất đồng bộ” nghĩa là client không phải đứng yên chờ toàn bộ việc kết thúc; nó có thể nhận ID, theo dõi trạng thái rồi lấy kết quả.

start_all_servers.ps1

Script PowerShell khởi động các agent server trên đúng port. Nó giải quyết phần vận hành local: mở nhiều process thay vì bắt người dùng gõ năm lệnh.

Process là một chương trình đang chạy. Mỗi server là một process riêng nên lỗi ở một service không nhất thiết làm tắt cả dừng.

Luồng chạy toàn hệ thống

1. Chạy script khởi động năm specialist server.
2. Mỗi specialist công bố Agent Card.
3. Chạy adk web cho orchestrator ở port 8000.
4. Người dùng gửi yêu cầu.
5. Root agent đọc intent, tức mục đích của yêu cầu.
6. Root agent chọn specialist theo description.
7. RemoteA2aAgent gửi message đến endpoint tương ứng.
8. Specialist chạy workflow nội bộ và dùng tool riêng.
9. Kết quả quay về orchestrator.
10. Orchestrator trả lời người dùng.

Ví dụ routing

“Lên lịch đi Huế ba ngày” → Trip Planner.

“Đọc brief này và sửa kịch bản” → Script Team.

“So sánh hiệu suất hai đội ở World Cup” → World Cup Analyst.

“Phân tích gu bạn đời của tôi” → Love Advisor.

“Đọc file doanh thu và tìm insight” → Dashboard Insights.

Nếu một yêu cầu có nhiều domain, orchestrator có thể chia việc cho nhiều specialist rồi tổng hợp. Nhưng khả năng này cần instruction và kiểm thử rõ; không nên mặc định tuyên bố đã hỗ trợ hoàn hảo.

Điểm khác biệt

A2A Orchestrator chuyển kiến trúc từ “nhiều agent trong một process” sang “nhiều hệ agent độc lập qua mạng”.

Lợi ích:

- Mỗi team có thể phát triển agent riêng.
- Mỗi agent dùng dependency riêng.
- Có thể scale specialist nặng độc lập.
- Có thể thay một agent mà không viết lại tất cả.

Chi phí:

- Có lỗi mạng và timeout.
- Cần authentication.
- Cần versioning Agent Card và protocol.
- Cần tracing xuyên service.
- Khó debug hơn code chạy trong một process.

Lưu ý phiên bản

Tài liệu ADK hiện mô tả tích hợp A2A Python là experimental và hỗ trợ cả dòng A2A SDK 0.3.x lẫn 1.x.x. Tích hợp mới nên hướng đến 1.x.x.

Vì giao thức phát triển theo thời gian, không nên đóng đinh một endpoint hoặc tên method cũ nếu chưa kiểm tra phiên bản đang cài. Điều đáng tin hơn là:

1. Đọc Agent Card thật.
2. Kiểm tra version.
3. Dùng SDK tương thích.
4. Chạy smoke test end-to-end.

Nâng cấp hợp lý

- Health check cho từng specialist.
- Timeout, retry có backoff và circuit breaker.
- Authentication giữa các service.
- Trace ID đi xuyên orchestrator và specialist.
- Registry động thay vì hard-code URL.
- Capability-based routing có evaluator.
- Fallback khi specialist không online.

backoff là tăng dần thời gian chờ giữa các lần thử lại. circuit breaker là cơ chế tạm ngừng gọi một service đang lỗi liên tục để tránh làm hệ thống tệ hơn.

Câu chốt

A2A không làm agent thông minh hơn. Nó làm nhiều agent có thể làm việc cùng nhau mà vẫn giữ ranh giới độc lập.

13. Ba mẫu multi-agent cần phân biệt

Mẫu	Project	Ai giữ quyền điều khiển?	Khi nào dừng?
Transfer	Script Team	Agent được chuyển giao	Khi vai trò sau cần tiếp quản hội thoại
Agent as Tool	World Cup Analyst	Root agent	Khi root muốn gọi chuyên gia rồi nhận kết quả
Workflow Agents	Love Advisor, Dashboard Insights	Workflow xác định trước	Khi thứ tự, song song hoặc vòng lặp phải rõ
Remote Agent qua A2A	Orchestrator	Orchestrator + giao thức	Khi agent chạy như service độc lập

Không có mẫu nào luôn tốt nhất. Chọn theo quyền điều khiển, mức độc lập và độ chắc của workflow.

14. Ma trận toàn bộ hệ project

Project	Bài học chính	Tool/Data	State/Output	Kiến trúc
Trip Planner	Tool + state	save/read trip	Session state	Single Agent
Script Team	Chuyên môn hóa	file reader, memory	Draft/feedback/revision keys	Transfer team
World Cup Analyst	Research + computation	Wikipedia, code executor	last_wc_report	AgentTool
Love Advisor	Song song + tổng hợp	web search, frameworks	PartnerPlan JSON	Parallel inside Sequential
Dashboard Insights	Pipeline đáng tin	file ingestion, MCP, RAG, logging	reports, archive, vector store	Sequential + Loop
A2A Orchestrator	Hệ sinh thái phân tán	Agent Cards, remote calls	remote responses/tasks	RemoteA2aAgent

15. Lộ trình năng lực kỹ thuật

Nấc 1 — Một agent biết gọi hàm

Trip Planner: model, instruction, tools, state.

Nấc 2 — Nhiều vai trò

Script Team: chuyển giao và output keys.

Nấc 3 — Chuyên gia như một công cụ

World Cup Analyst: search agent và stats agent được bọc thành tools.

Nấc 4 — Workflow có hình dạng

Love Advisor: song song, tuần tự, schema.

Nấc 5 — Pipeline có dữ liệu và QA

Dashboard Insights: ingestion, MCP, RAG, loop, logging, archive.

Nấc 6 — Hệ phân tán

A2A Orchestrator: agent discovery, remote service, routing, port và protocol.

Lộ trình này là câu chuyện mạnh nhất để đưa vào CV: không phải “tôi làm sáu chatbot”, mà là “tôi khảo sát sáu mức kiến trúc agent”.

16. Checklist build một agent từ đầu

Trước khi code

- Agent giải quyết một việc cụ thể nào?
- Việc nào model làm, việc nào tool làm?
- Dữ liệu nào cần nhớ trong session?
- Kết quả cần là văn bản hay schema?
- Hành động nào cần xác nhận?
- Thành công được đo bằng test nào?

Khi tạo file

- `.env.example` có đủ tên biến.
- `.gitignore` chặn secret và dữ liệu local.
- `requirements.txt` khớp import thật.
- README có lệnh cài và chạy.
- Tool có type hint và description rõ.
- Test tool riêng trước.

Khi nối model

- Instruction nói rõ mục tiêu, quy trình và điều cấm.
- Tool schema không mơ hồ.
- Có timeout và xử lý lỗi.
- Không để model tự tuyên bố tool đã chạy.
- Có log cho tool call.

Khi mở rộng multi-agent

- Mỗi agent chỉ có một vai trò rõ.
- Quyết định dùng transfer, AgentTool hay workflow.
- Output của agent trước có hợp đồng cho agent sau.
- Có giới hạn loop.
- Có cách xem trace.

Khi đưa qua A2A

- Mỗi agent có Agent Card đúng.
- Port không trùng.
- Version SDK/protocol tương thích.
- Có health check.
- Có timeout và fallback.
- Không đưa secret vào Agent Card.

17. Những điểm nên nâng cấp trước khi gọi là production

Secrets và quyền truy cập

Không commit .env. Dùng secret manager khi deploy. Secret manager là dịch vụ lưu khóa bí mật và chỉ cấp cho process được phép.

Data validation

Schema đầu vào, schema đầu ra và kiểm tra business rule. Type đúng chưa đủ; ví dụ ngân sách là số dương mới hợp lý.

Observability

Log, metric và trace.

- Log kể từng sự kiện.
- Metric là số tổng hợp như tỉ lệ lỗi hoặc độ trễ.
- Trace nối toàn bộ các bước của một request.

Evaluation

Tạo test set có expected behavior. Không chỉ hỏi vài câu “trông có vẻ ổn”.

Đánh giá nên có:

- task success,
- tool selection,
- factuality,
- format validity,
- safety,
- latency,
- cost.

Human-in-the-loop

Cho người dùng xác nhận trước hành động nhạy cảm, tốn tiền hoặc không thể hoàn tác.

Persistence

In-memory để học. Database để giữ dữ liệu. Vector database để tìm theo ngữ nghĩa. Mỗi loại giải quyết một bài khác nhau.

Security cho tool

Giới hạn đường dẫn file, domain mạng, code execution và kích thước input. Tool là nơi agent chạm thế giới thật; đây cũng là nơi rủi ro đi vào.

18. Glossary — thuật ngữ nói đơn giản

Agent

Chương trình dùng model để quyết định và có thể thực hiện hành động qua tool.

LLM

Mô hình ngôn ngữ lớn, phân tạo và hiểu văn bản.

Prompt

Nội dung gửi cho model trong một lượt.

Instruction

Quy tắc làm việc dài hạn của agent.

Tool

Hàm hoặc dịch vụ agent có thể gọi.

Runtime

Phần chương trình thực sự vận hành agent và gọi tool.

State

Dữ liệu tạm có cấu trúc trong một phiên.

Session

Một luồng hội thoại gồm lịch sử, state và metadata.

Memory

Thông tin có thể được truy xuất lại ngoài một lượt hoặc một session.

Schema

Hợp đồng mô tả dữ liệu có những trường gì và kiểu gì.

Callback

Hàm code được gọi tự động trước hoặc sau một mốc chạy.

Planner

Thành phần giúp agent chia nhiệm vụ thành các bước.

Structured output

Kết quả tuân theo cấu trúc định trước, ví dụ JSON.

RAG

Tìm đoạn dữ liệu liên quan rồi cho model đọc trước khi trả lời.

Embedding

Dãy số biểu diễn ý nghĩa để máy so sánh độ gần ngữ nghĩa.

Vector database

Kho dữ liệu tối ưu cho tìm kiếm theo độ gần giữa embedding.

MCP

Chuẩn kết nối agent với tool và nguồn dữ liệu.

A2A

Chuẩn giao tiếp giữa các agent độc lập.

Agent Card

Hồ sơ công khai năng lực và cách gọi một A2A agent.

Endpoint

Địa chỉ mạng mà client gửi request tới.

Port

Số cổng giúp phân biệt nhiều server trên cùng một máy.

Artifact

Sản phẩm do agent tạo ra, như report hoặc file.

Sandbox

Môi trường giới hạn quyền chạy code để giảm rủi ro.

Guardrail

Quy tắc hoặc code chặn hành vi không mong muốn.

A2A Orchestrator

Agent điều phối, chọn đúng agent chuyên gia và chuyển yêu cầu qua A2A.

19. Prompt gợi ý cho NotebookLM

Học nhanh

“Dùng ví dụ trong sáu project để giải thích agent khác chatbot. Không dùng thuật ngữ nào mà không giải thích.”

Tạo Audio Overview

“Tạo cuộc trò chuyện hai người. Người thứ nhất là sinh viên mới học. Người thứ hai là kỹ sư giải thích hành trình từ Trip Planner đến A2A Orchestrator. Mỗi project phải có một bài học và một giới hạn.”

Tạo Mind Map

“Tạo mind map lấy AI Agent Ecosystem làm gốc, chia nhánh Foundation, Tools, State, Workflow, Data, QA, MCP và A2A.”

Tạo câu hỏi phỏng vấn

“Tạo 20 câu hỏi phỏng vấn intern dựa trên project, kèm câu trả lời ngắn và câu hỏi đào sâu.”

Tạo nội dung CV

“Viết 5 bullet CV. Chỉ dùng tính năng đã có trong code. Không bịa metric hay production deployment.”

Tạo short video

“Tạo chuỗi 12 video 45 giây. Mỗi video có hook, giải thích, ví dụ project và câu chốt.”

20. Thẻ kịch bản video ngắn

Short 01 — Chatbot và Agent khác gì?

Hook: Cùng dùng AI, nhưng một bên chỉ nói. Một bên biết làm.

Giải thích: Chatbot trả lời một lượt. Agent có vòng lặp quan sát, suy nghĩ, gọi tool rồi nhìn kết quả.

Ví dụ: Trip Planner không chỉ viết lịch trình. Nó gọi tool để lưu và đọc lại lịch trình.

Câu chốt: Có hành động, có quan sát lại, mới bắt đầu giống agent.

Short 02 — Vì sao cần agent.py?

Hook: Nếu project agent là một đội, agent.py là bản phân công.

Giải thích: File này khai báo tên agent, model, instruction, tools và các agent con.

Câu chốt: Não nào, luật gì, tay nào. Nhìn agent.py là thấy.

Short 03 — .env để làm gì?

Hook: API key không thuộc về GitHub.

Giải thích: .env giữ cấu hình bí mật trên máy. .env.example chỉ giữ tên biến để người khác biết cần chuẩn bị gì.

Câu chốt: Một file để chạy. Một file để hướng dẫn. Đừng nhầm.

Short 04 — Tool chạy như thế nào?

Hook: Model không tự chạy Python.

Giải thích: Model chọn tool và tạo tham số. Runtime kiểm tra rồi gọi hàm thật. Kết quả quay lại model.

Câu chốt: Model quyết định. Tool thực thi.

Short 05 — State khác memory?

Hook: Nhớ trong cuộc nói chuyện và nhớ qua nhiều cuộc nói chuyện là hai việc.

Giải thích: State là bảng nháp của session hiện tại. Memory là kho có thể truy xuất lâu hơn.

Ví dụ: Trip Planner dùng state; Script Team thử memory service.

Short 06 — Vì sao phải test tool trước?

Hook: Agent sai chưa chắc do AI.

Giải thích: Có thể tool sai, schema sai hoặc instruction sai. Test riêng giúp khoanh vùng.

Câu chốt: Tách lỗi trước. Nối não sau.

Short 07 — Một agent hay một team?

Hook: Prompt càng dài không có nghĩa agent càng giỏi.

Giải thích: Script Team chia Drafter, Critic và Reviser.

Câu chốt: Chia đúng vai. Sau này đỡ rối.

Short 08 — Transfer khác AgentTool?

Hook: Giao hẳn việc và gọi chuyên gia tư vấn không giống nhau.

Giải thích: Transfer chuyển quyền xử lý. AgentTool để root gọi chuyên gia rồi lấy kết quả lại.

Ví dụ: Script Team dùng transfer; World Cup Analyst dùng AgentTool.

Short 09 — ParallelAgent để làm gì?

Hook: Hai việc không phụ thuộc nhau thì đừng bắt chúng xếp hàng.

Giải thích: Love Advisor phân tích ngoại hình và tính cách song song rồi tổng hợp tuần tự.

Câu chốt: Độc lập thì chạy song song. Phụ thuộc thì chạy tuần tự.

Short 10 — Structured output có gì hay?

Hook: Đoạn văn đọc được. JSON còn dùng được.

Giải thích: Pydantic ép kết quả theo schema. Giao diện có thể lấy từng trường để hiển thị.

Câu chốt: Format rõ là hợp đồng giữa AI và phần mềm.

Short 11 — RAG là gì?

Hook: Đừng bắt model nhớ tất cả. Hãy tìm đúng đoạn cho nó đọc.

Giải thích: RAG tìm văn bản gần nghĩa bằng embedding rồi đưa vào context.

Ví dụ: Dashboard Insights dùng ChromaDB và sentence-transformers.

Short 12 — MCP khác A2A?

Hook: Tool và đồng nghiệp không phải một thứ.

Giải thích: MCP nối agent với tool/data. A2A nối agent với agent.

Câu chốt: MCP là ổ cắm. A2A là cuộc gọi nội bộ.

Short 13 — LoopAgent có nguy hiểm không?

Hook: Cho AI tự sửa mãi là một vòng lặp không có phanh.

Giải thích: Dashboard Insights giới hạn QA–revision tối đa hai vòng.

Câu chốt: Loop phải có tiêu chí dừng và giới hạn.

Short 14 — Agent Card là gì?

Hook: Agent cũng cần một tấm danh thiếp.

Giải thích: Agent Card nói agent tên gì, làm được gì, nhận dữ liệu nào và gọi ở đâu.

Câu chốt: Card rõ thì orchestrator mới gọi đúng người.

Short 15 — Port 8000 đến 8005 để làm gì?

Hook: Sáu agent cùng một máy nhưng không chen cùng một cửa.

Giải thích: Mỗi server dùng một port. Orchestrator ở 8000, năm specialist ở 8001–8005.

Câu chốt: Cùng địa chỉ máy. Khác số phòng.

Short 16 — Vì sao cần logging?

Hook: Agent trả sai mà không có log thì chỉ còn đoán.

Giải thích: Log ghi tool nào chạy, input gì và kết quả ra sao.

Câu chốt: Không quan sát được thì chưa vận hành được.

Short 17 — Hạn chế cũng là nội dung portfolio

Hook: Project tốt không cần giả vờ hoàn hảo.

Giải thích: Nêu rõ in-memory mất dữ liệu, RAG có thể nhiễm domain, A2A cần versioning và retry.

Câu chốt: Biết hệ thống yếu ở đâu là dấu hiệu của tư duy kỹ sư.

Short 18 — Câu chuyện của cả hệ thống

Hook: Đây không phải sáu chatbot.

Giải thích: Trip Planner dạy tool và state. Script Team dạy phân vai. World Cup dạy research và code. Love Advisor dạy workflow. Dashboard dạy data và QA. Orchestrator nối tất cả bằng A2A.

Câu chốt: Một agent làm việc. Một hệ agent phối hợp.

21. Kết luận

Nếu chỉ nhìn model, mọi project agent trông giống nhau. Nếu nhìn vào kiến trúc, chúng khác rất xa.

- Tool quyết định agent chạm được gì.
- State và memory quyết định agent nhớ được gì.
- Workflow quyết định việc đi qua những bước nào.
- Callback và logging quyết định ta kiểm soát được gì.
- MCP quyết định agent kết nối tool ra sao.
- A2A quyết định các agent độc lập hợp tác thế nào.

Đó là câu chuyện xuyên suốt của bộ project: bắt đầu từ một hàm lưu lịch trình, kết thúc ở một orchestrator điều phối năm chuyên gia qua mạng.

Không cần gọi nó là “siêu AI”.

Chỉ cần chỉ ra: hệ thống nhận gì, quyết định gì, gọi gì, lưu gì, kiểm tra gì và trả ra gì.

SOURCE-BASED • LIMITATIONS DISCLOSED • NO FABRICATED METRICS